SO-101 による位置可変 Pick & Place と、柔らかさの分からない物体の把持力制御

Physical AI 応用編 1 最終課題 説明資料　omnicampus アカウント名

コード: https://github.com/Natsu-B/Learn-SO-101-basic　　動画: （YouTube の URL）（約 2 分、すべて等倍）

1 概要

MuJoCo 上の SO-101 アームに、二つのタスクを実行させた。 一つ目は課題の本体で、テーブル上の 25 mm 立方体を掴み、目標位置まで運んで置く。 立方体の初期位置と目標位置は試行ごとに乱数で変わる。 掴む動作と置く動作は逆運動学（IK）で制御し、運ぶ動作は強化学習で得た方策（観測から関節の指令を出すニューラルネットワーク）の推論で制御した。 初期位置の異なる 200 試行すべてで配置に成功し、置いた位置の誤差は平均 2.42 mm、最大 6.42 mm だった。

二つ目は独自に追加した課題で、柔らかさと摩擦と質量が分からない物体を、指定した力で潰さずに掴む。触覚による把持力制御を強化学習で学習した論文 [1] を調べ、SO-101 に移植した。 その過程で、SO-101 の片側しか動かないグリッパだけでは、物体を固定側のジョーまで押し寄せる間に柔らかい物体が潰れることが分かった。そこで腕の横移動とグリッパを同時に動かす方策を学習させ、潰さずに目標力で掴めた試行の割合を、グリッパだけの最良の制御の 19.0 % から 92.0 % に上げた。

工夫した点は二つある。 一つは、動作ごとの成功判定を一か所で定義し、学習と評価と動画で同じ判定を使ったことである。 もう一つは、論文の手法が SO-101 で失敗する原因を計測で突き止め、腕の微小移動で補う方法に作り替えたことである。 数値はすべて、リポジトリにある評価スクリプトの出力から書いた。 実機は使っていない。

2 タスクとシステムの構成

2.1 シミュレータとソフトウェア

物理計算とタスク管理は Rust で書き、強化学習だけを Python の Stable-Baselines3（以下 SB3）で行った（図 1）。Rust 側は MuJoCo のモデル読み込み、接触の判定、IK、Skill（掴む、運ぶ、置くの各動作）、成功判定を持つ。 Python とは PyO3 で接続し、Gymnasium 環境として SB3 に渡す。 制御周期は 30 Hz、物理計算の刻みは 5 ms で、SO-101 モデル（MuJoCo Menagerie）の関節範囲とアクチュエータの力の上限はそのまま使った。 立方体の位置をプログラムから書き込むのはリセット時だけで、その後は接触の物理だけで動く。

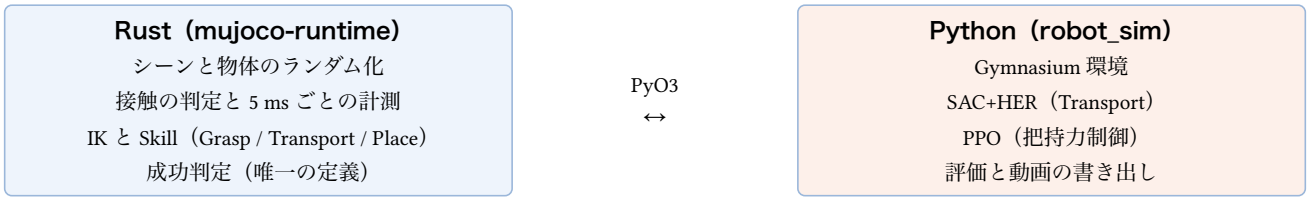


図 1: システムの構成。成功判定は Rust 側で一度だけ定義し、学習、評価、動画で共用する。

2.2 Pick & Place の手順と成功判定

立方体（25 mm、30 g）と目標は、どちらもアーム根元から半径 0.15〜0.25 m、方位±0.8 rad の範囲に、互いに 0.08 m 以上離して置く。 立方体の向き（yaw）は 0 に固定した。制御にはシミュレータから得た立方体と目標の位置の真値を使い、画像認識は扱っていない。 一つの試行は Grasp、Transport、Place の三つの Skill を順に実行し、前の Skill の成功判定が通ったら次へ進む。前の Skill の終了状態が次の Skill の初期状態になるので、Transport の学習でも、実際に Grasp を実行した後の状態から開始した。 各 Skill の成功判定を 表 1 に示す。

Skill	成功の条件
Grasp（IK）	両ジョーが立方体に接触し、テーブルから離れ、立方体中心が静止時より 30 mm 以上高い状態が 0.3 s 続く
Transport（SAC）	両ジョーで挟んだまま、立方体中心が静止時より 15 mm 以上高く、目標との XY 距離が 15 mm 以内
Place（IK）	ジョーが離れてテーブルに接し、高さの誤差 3 mm 未満、速さ 0.01 m/s 未満、目標との XY 距離 20 mm 以内

表 1: 成功判定（Rust の `predicates`）。学習環境、評価、動画で同じ定義を使う。

2.3 学習済みモデルを使った箇所

Transport は、SAC（連続値の行動を学習する強化学習手法）と HER [2]（失敗した経験の目標を、実際に到達した位置に置き換えて再利用する手法）で学習した方策を、毎制御周期に推論して動かす。観測は腕 5 関節の角度と角速度、挟持中心と立方体の位置、立方体の速度、グリッパ角など 23 次元、行動は腕 5 関節の角度の増分で、グリッパは閉じたまま保つ。 報酬は立方体と目標の距離だけで決めた。 学習は 5 万 step で止め、学習中の評価で最も良かった

3 万 step 時点のモデルを使った（このモデルは Transport 単体の評価で 20/20 成功）。発展課題の方策は、別の強化学習手法である PPO [3] で学習した（節 4）。

3 位置可変 Pick & Place の結果

seed 0〜199 の 200 試行で評価した（表 2）。学習した Transport を使った構成は 200 試行すべてで成功し、立方体の落下は 0 件だった。同じ seed 列で全 Skill を IK にした構成も 200/200 で、最終誤差の平均はほぼ同じ（2.43 mm）である。学習した Transport は、IK で書いた運搬と同等の結果を、学習済み方策の推論で出せた。図 2 に動画の一場面を示す。

構成	全体	Grasp	Transport	Place	最終 XY 誤差 平均 / P95 / 最大
IK → **SAC（学習）** → IK	200/200	200/200	200/200	200/200	2.42 / 3.43 / 6.42 mm
すべて IK	200/200	200/200	200/200	200/200	2.43 / 3.94 / 6.95 mm

表 2: Pick & Place の評価（seed 0〜199、各 200 試行。落下はどちらも 0 件）。

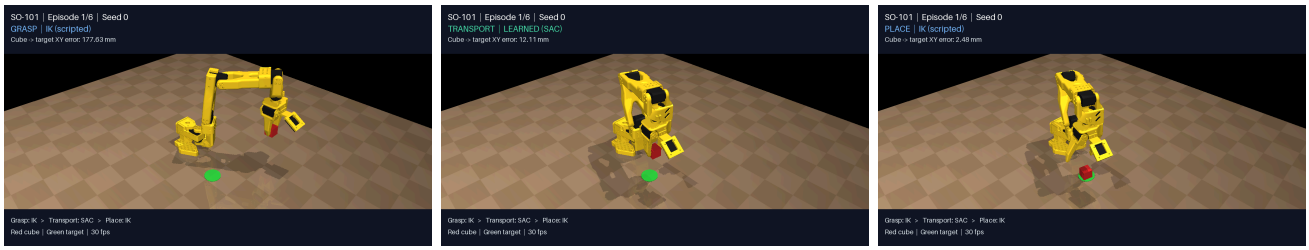


図 2: 動画の場面（左から Grasp、学習した Transport、Place）。画面上部に、実行中の Skill と、学習済みモデルか IK かを表示した。

開発中に実際に起きた失敗と、その原因は次の二つだった。IK の姿勢誤差に外積を使うと、指先が真上を向く解に収束したので、方向ベクトルの差に変えた。固定ジョーの内面と立方体の側面のすき間は約 1 mm しかなく、下降中に立方体の上面へ当たることがあった。下降中だけ固定ジョーを 4 mm 離し、閉じる動作で立方体を押し付けるようにした。最初はずらす向きを逆にしてしまい、成功が 95/200 まで落ちたので、ログで確かめて直した。

物理の確認はコンパイルと分けてテストにした。両ジョーで挟めば 30 mm 以上持ち上がり、同じ動作でグリッパを閉じなければ持ち上がらない、という対照実験を Rust のテストに入れてある。動画の描画は物理状態をコピーして行い、描画の有無で軌道が変わらないこと（seed 0 で完全一致）を確認した。

4 柔らかさの分からない物体の把持力制御

4.1 取り組んだ理由と実験の条件

本体の Pick & Place では、グリッパに−0.17 rad の閉指令を与え続けるため、把持中のアクチュエータのトルクが上限の 2.94 N·m に達し、立方体を片側約 39 N で握っていた。重さ 30 g の立方体に対して大きな力で、柔らかい物体を扱うなら、物体が耐えられる力に合わせて握る力を調整する必要がある。未知の質量と摩擦に対して掴み方を学習させる実験（残差強化学習）も行ったが、IK や固定の補正に勝てなかった（節 5）。そこで、握る力そのものを接触力のフィードバックで制御する実験に切り替えた。

この実験は、Pick & Place とは別の独立した環境で行った。アームを物体の真上に下ろした状態から始め、テーブル上の模擬物体を 5 s（150 制御周期）掴む。物体は持ち上げない。目標力は試行ごとに、物体が耐えられる力（剛性×最大圧縮量）の 80 % 以下かつ 8 N 以下の範囲から選び、観測として方策に与える。物体の剛性（200〜5000 N/m）、幅、ジョーとのすき間、摩擦、質量も試行ごとに変えるが、こちらは方策に与えない。

4.2 論文の調査と、柔らかい物体のモデル

参考にしたのは、TIAGo の 2 指グリッパで、触覚センサの力を見ながら PPO で把持力を制御した研究 [1] である。調査と再実装には、公開されている arXiv 初版（2311.07245v1、2023 年）を用いた。公開実装にはライセンスの表記がなかったので、コードは写さず、論文の式と設定値をもとに実装し直した。論文と公開コードには食い違いがあり、たとえば論文の主な手法は行動を接触状態に応じて縮める仕組み（帰納バイアス）を使うが、公開の学習スクリプトではこれが無効で、物体の移動に対する罰の重みも 0 だった。論文の柔らかさは、MuJoCo の数値的なめり込みとセンサ値の係数で表されていて、物体の変形状態はモデルに存在しない。

この点を変え、物体は剛体の芯の 4 側面に、ばねとダンパ付きのスライド関節で板を付けたモデルにした（図 4 の青い部分）。板の押し込み量を d、押し込み速度を $\dot{d}$、ばね定数を k、減衰係数を c とすると、板の復元力は $F = kd + c\dot{d}$ になる。押し込みが上限に達した状態を潰れ、ジョーが板に 1 mm を超えてめり込んだ状態を無効な接触として、変形とは別に記録した。静的な保持試験では、潰れる前のジョーの法線力とばねとダンパによる力の差が 0.05 N 以内だった。接触力の最大値は、30 Hz の制御周期の終わりだけでなく、5 ms の物理ステップごとに記録した。衝突の瞬間の力は周期の終わりの値の 2 倍以上になることがあり、周期の終わりだけを見ると見落とす。

成功は、両ジョーの接触を獲得し、試行中に潰れと無効な接触がなく、最後の 1 s で接触を失わず、その間の平均力の誤差が目標力の 10 % と 0.05 N の大きい方以内に収まった試行とした。

4.3 グリッパだけでは潰れる理由

論文の公開学習スクリプトに合わせ、物体移動の罰と帰納バイアスを外した PPO は、力の平均追従誤差が 0.208 N で、SO-101 向けに調整した力 PI 制御（論文のベースラインと同じ構造）の 0.278 N を下回った。論文どおりに罰と帰納バイアスを入れた設定では、誤差は 2.70 N だった。 どちらの設定でも、潰さずに目標力で掴めた試行は 15 % 前後で、力 PI の 16.5 % と変わらなかった（図 3）。

失敗を段階に分けて数えると、潰れの多くは目標力に届く前、可動ジョーが物体を固定ジョーまで押し寄せる間に起きていた。SO-101 は片側のジョーしか動かないので、物体が可動ジョー側に寄っていれば、テーブル上を押して運ぶしかない。 この押し寄せに必要な力を測った。 可動ジョーをゆっくり閉じても、物体が滑り出すときの力はテーブル摩擦からの見積もりの中央値 1.72 倍あり、22 % の物体では板が耐えられる力を超えた。 回転するジョーが物体を斜め下にも押し、摩擦を増やしているためと考えている。ゆっくり押しても 36 % の試行で板が潰れたので、グリッパの制御だけでは避けきれない失敗と判断した。報酬を変えた学習（ピーク力の罰、両ジョー接触の獲得に対する報酬）や、押す力の上限を目標力の 0.8 倍から毎秒 0.5 N ずつ上げるスクリプト制御（二段階 PI）も試したが、成功率は最大 19 % だった。

4.4 腕とグリッパの協調

論文の 2 本指では、先に触れた指を止め、もう一方の指を寄せれば物体を押さずに済む。 SO-101 では、腕を把持方向へ数 mm 動かせば、固定ジョーを物体に寄せられる。 そこで腕の目標位置を把持方向へ最大 12 mm ずらす操作を Rust に追加し、まずスクリプト制御（腕補正 PI）で試すと、成功率は 44.0 % になった。

次に、グリッパの指令と腕の移動量を同時に出す 2 次元の方策を PPO で学習させた。 報酬は、両ジョーが接触している間だけ力の追従を評価し、両ジョーの接触の獲得に+20、試行の最後の安全把持の成功に+50、潰れと無効な接触にそれぞれ−10 を与えた。一度も両ジョーで接触しなければ−20 とし、接触を避けて罰を逃れる方策が得をしないようにした。50 万 step で成功率 67.5 % になったので、累積約 100 万 step まで延長した。選択用の 48 試行で最良だった約 70 万 step 時点のモデルを、評価用の別の 200 試行で評価すると 92.0 % だった（図 3、表 3、図 4）。 同じ 200 試行で成否を対応させると、腕補正 PI より成功した試行が 100 件、逆が 4 件で、McNemar 検定で $p = 4.7 \times 10^{-25}$ だった。 腕だけを学習し、握る力は PI に任せた構成でも 72.0 % になり、改善の多くは腕を動かす時機と速さの学習から来ている。

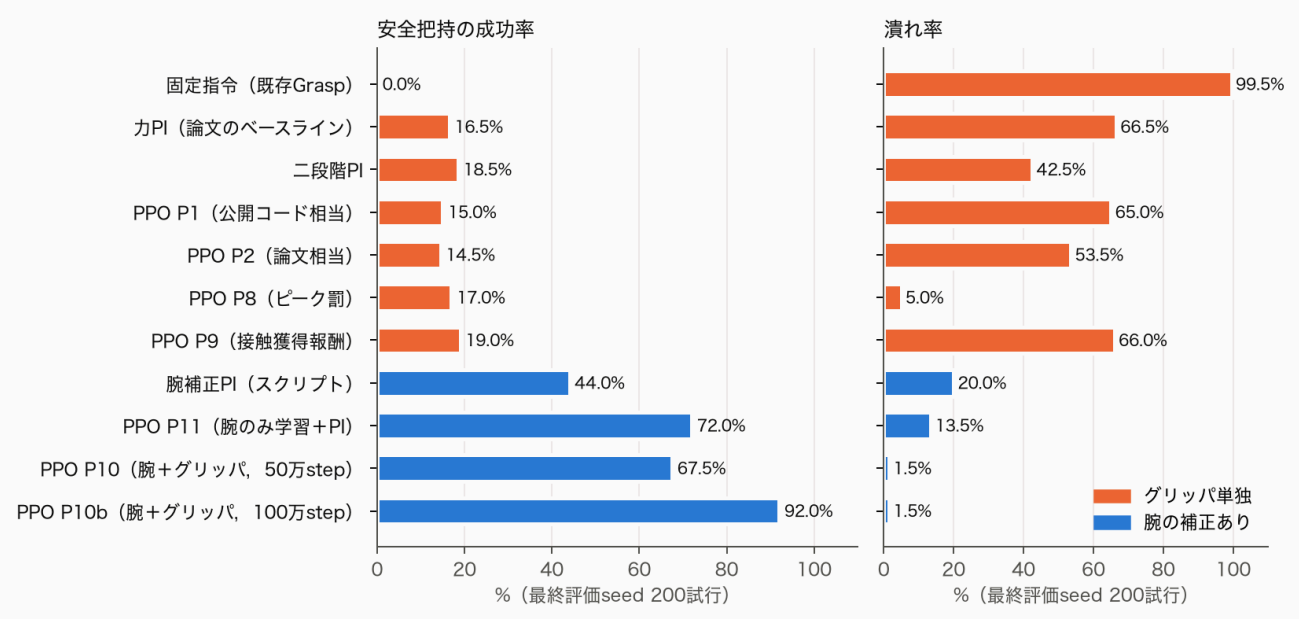


図 3: 潰さずに目標力で掴めた割合（安全把持の成功率）と潰れ率。評価用の 200 試行（seed 8,000,000〜8,000,199）で、全制御が同じ物体、目標力、センサノイズを使う。PPO P9 は両ジョー接触の獲得を報酬に加えたグリッパだけの方策、P10 と P10b は腕とグリッパの 2 次元の方策（50 万 step と延長後）、P11 は腕だけを学習しグリッパは PI の構成。

制御（200 試行）	成功	潰れ	無効な接触	物体の移動	ピーク力 中央値 / P95
力 PI（グリッパのみ）	16.5 %	66.5 %	30.0 %	7.3 mm	14.4 / 33.7 N
PPO P9（グリッパのみ）	19.0 %	66.0 %	34.5 %	7.7 mm	11.4 / 45.2 N
腕補正 PI（スクリプト）	44.0 %	20.0 %	21.5 %	1.8 mm	2.5 / 8.4 N
PPO P10b（腕＋グリッパ）	**92.0 %**	1.5 %	5.5 %	1.1 mm	2.2 / 8.2 N

表 3: 把持力制御の比較。ピーク力は、両ジョーで接触し、無効な接触がなかった試行だけで集計した。

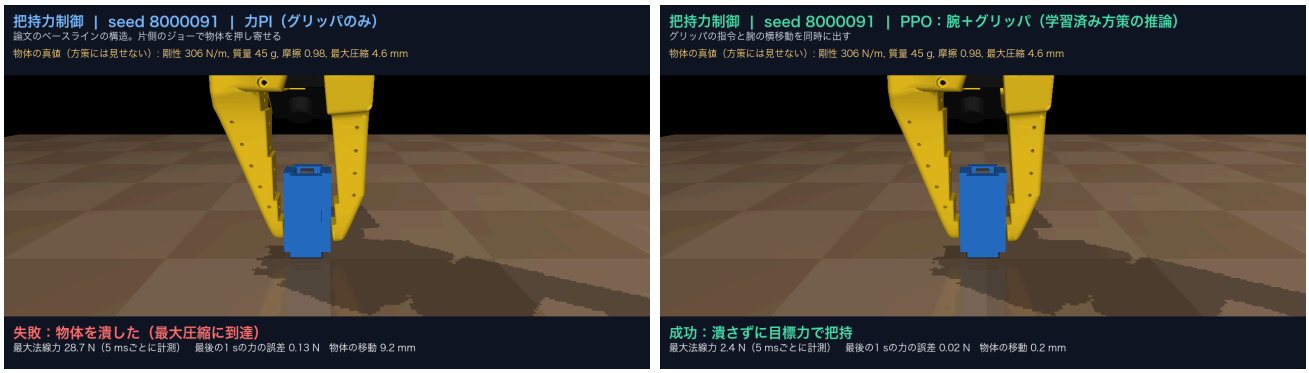


図 4: 同じ柔らかい物体（剛性 306 N/m、目標力 0.68 N）での比較。左の力 PI は押し寄せる間に物体を潰し（最大 28.7 N）、右の腕とグリッパの PPO は潰さずに掴んだ（最大 2.4 N）。

選択用の成功率はチェックポイントごとに大きく揺れ、延長した学習では 0.15〜0.90 の間を動いた（図 5）。同じ学習の最後のモデルは評価用の 200 試行で 78.5 % で、どのチェックポイントを選ぶかで成績が変わる。学習はまだ安定していない。

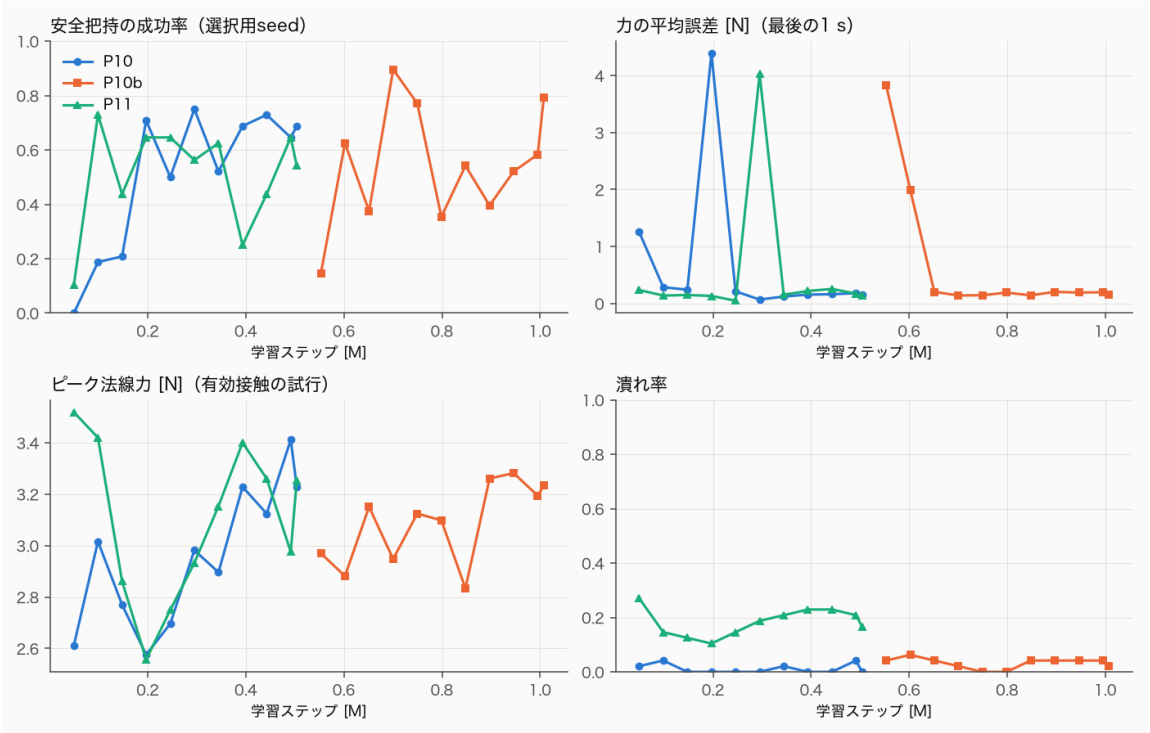


図 5: 腕とグリッパの協調学習の推移（選択用の 48 試行）。P10 は 50 万 step、P10b は P10 を延長した学習、P11 は腕だけを学習した構成。

5 うまくいかなかったことと限界

未知の質量と摩擦に対して、IK の Grasp に学習した補正を足す実験を行った。質量 20〜100 g、摩擦係数 0.05〜1.5 で学習し、学習範囲外の 120 試行を加えた計 570 試行で評価した。全工程の成功数は、最初の報酬で 192〜237、後段の成功を含めた報酬で 222〜235 で、IK のみの 240 と手首の固定補正の 253 を下回った。この比較は、摩擦を変えると接触の次元と硬さも変わる最初の接触環境で行った。接触物理を調べると、軽い物体ほど指先が深くめり込むなど、MuJoCo の接触モデルに由来する偏りがあった。手首の固定補正も、値を変えたときの差は並べ替え検定で偶然のばらつきと区別できなかった。

把持力制御では、論文どおりの設定がそのままでは働かなかった。帰納バイアスを 1 自由度に合わせると、物体を押し寄せる間の閉じる速さが 10 分の 1 になり、150 制御周期の間に両ジョーが触れなくなった。ピーク力に罰を与え

た設定では、物体に触れない行動が学習された。 両ジョー接触の獲得を報酬の達成条件にしたのは、この失敗を受けてのことである。

残っている限界は次のとおりである。 力の観測はシミュレーション上の接触力で、実機の SO-101 には指先の力センサがない。 実機で使うには、センサを付けるか、サーボの電流から力を推定する必要がある。 硬い物体では衝突時に板がジョーへめり込む無効な接触が 5.5 % 残り、これは物理モデル側の問題である。 学習は乱数 seed 1 つだけで行い、把持力制御は Pick & Place の本体にはまだ組み込んでいない。

6 再現方法と AI の利用

すべて Nix の開発環境（`nix develop`）で動く。 学習済みモデルは `runs/` に置き、Git では管理していない。 評価にはモデルと同じディレクトリの `config.json` が必要なので、下の手順で学習して用意する。 92.0 % のモデルは、下の 50 万 step の学習の最後のモデルから `--resume` で 50 万 step 追加学習したもので、延長学習と全比較の評価コマンドは `docs/FORCE_CONTROL.md` の§11 に記載した。

```
cargo test --workspace && maturin develop --release && pytest -q      # Rust 66件 / Python 181件
python examples/python/train_transport_sac_her.py --steps 250000 --seed 42 \
  --output runs/so101-transport/seed-42                                 # 5万stepで停止して使用
python examples/python/eval_pick_place.py --model runs/so101-transport/seed-42/best.zip \
  --episodes 200 --seed 0 --transport learned                          # 表2
python examples/python/train_force_grasp_ppo.py --output runs/force-grasp-coop/p10 --steps 500000 \
  --eval-freq 49152 --eval-episodes 48 --checkpoint-freq 98304 --no-bias --alpha-obj 0 0 \
  --reward coop_safe --action-mode arm_gripper --select success
python examples/python/eval_force_grasp.py --output runs/eval --episodes 200 --controllers pi \
  armpi:push_ramp=0.5,push_speed=0.3,push_ratio=0.8,shift_speed=0.012 \
  ppo:P10=runs/force-grasp-coop/p10/best.zip
```

課題の要件である AI の利用は、学習済み方策の推論である。 Pick & Place の Transport は SAC+HER の方策、把持力制御は PPO の方策が、毎制御周期に行動を出す。 開発では、Claude Code と Codex をコードの作成、実験の実行、文書の作成と点検に使った。 表の数値は評価スクリプトの出力から転記し、評価条件とログの保存先をリポジトリの `docs/` に記録した。

参考文献

[1] L. Lach *et al.*, “Zero-Shot Transfer of a Tactile-based Continuous Force Control Policy from Simulation to Robot,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 725–732. doi: 10.1109/IROS58592.2024.10802386.

[2] M. Andrychowicz *et al.*, “Hindsight Experience Replay,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” 2017.